Assignment 3

MATH_BASELINE

A. Category counts:
- Cat1 (format=1, answer=1) - 304
- Cat2 (format=1, answer=0) - 166
- Cat3 (format=0, answer=0) - 30

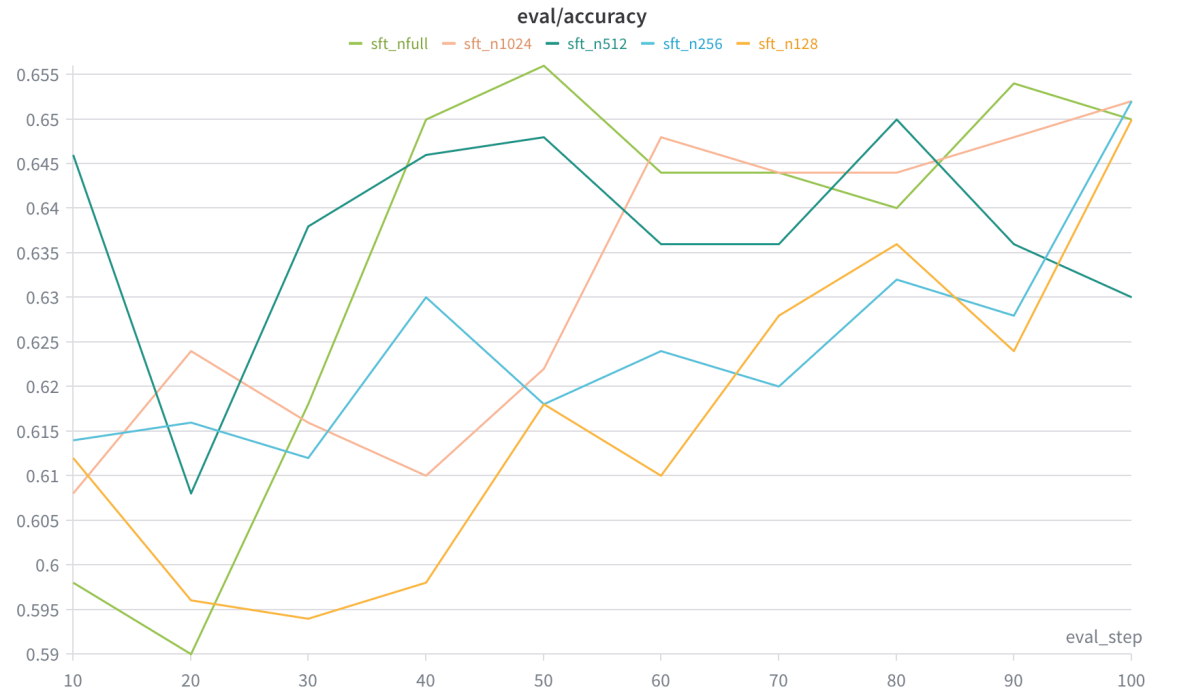
Looking at examples from Cat3, the model fails to conclude with the boxed answer as instructed. The model writes in plain text without LaTeX formatting at all (e.g. outputs "x ≈ 1.25" and "5/4" in prose), and the model runs out of tokens mid-reasoning before ever reaching a boxed conclusion seen in multiple examples 3, 7, 9, 10 all end abruptly mid-sentence at the 2048-token limit.

Cat2 the answer is mathematically wrong. These are genuine reasoning errors by the model Example (GT: √51) confuses which side is opposite vs. adjacent in a right triangle; (GT: 5) sets up the dog-walking ratio problem incorrectly; circular permutation counting wrong. The parser is not to blame it correctly extracts whatever is inside the box, and the model simply solved the problem incorrectly.

B. The Qwen 2.5 Math 1.5B zero-shot baseline achieves **60.8% accuracy** on the 500 example MATH test set using greedy decoding (temperature=0.0) with the Prime Intellect prompt, with the majority of failures (166/500) being correct-format but wrong-answer cases rather than formatting failures (30/500).

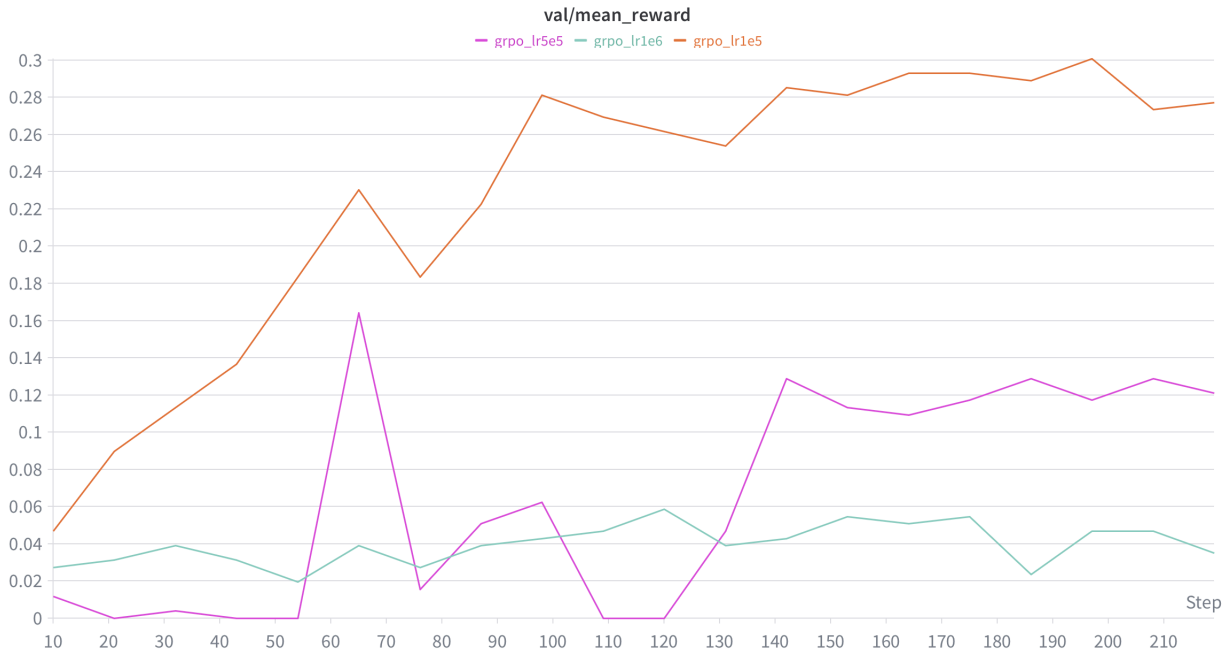
SFT_EXPERIMENT

1.



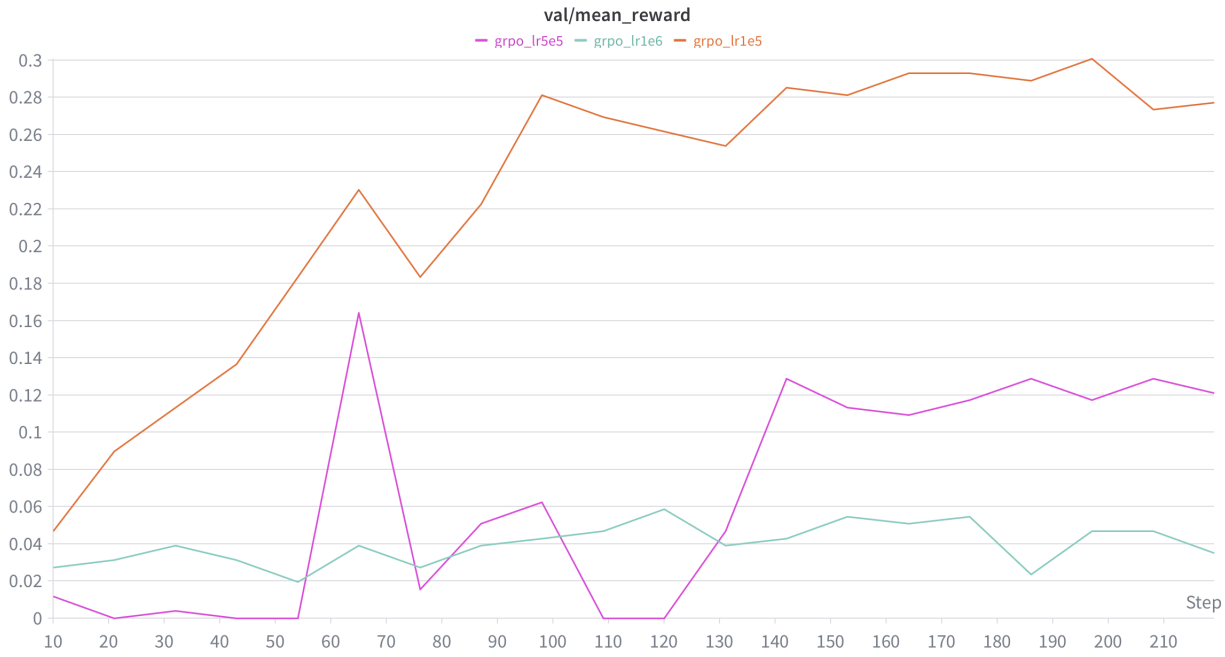
2. Our best SFT model (trained on the full Prime Intellect dataset with LR=1e-5) achieves **67.0% accuracy on the Prime Intellect test set** and **70.0% accuracy on the MATH test set**. The model correctly follows the required response format in 97% of cases (only 26/500 format failures on Intellect, 29/500 on MATH), indicating that SFT effectively teaches both the reasoning format and mathematical problem-solving ability.

GRPO_TRAIN_LOOP



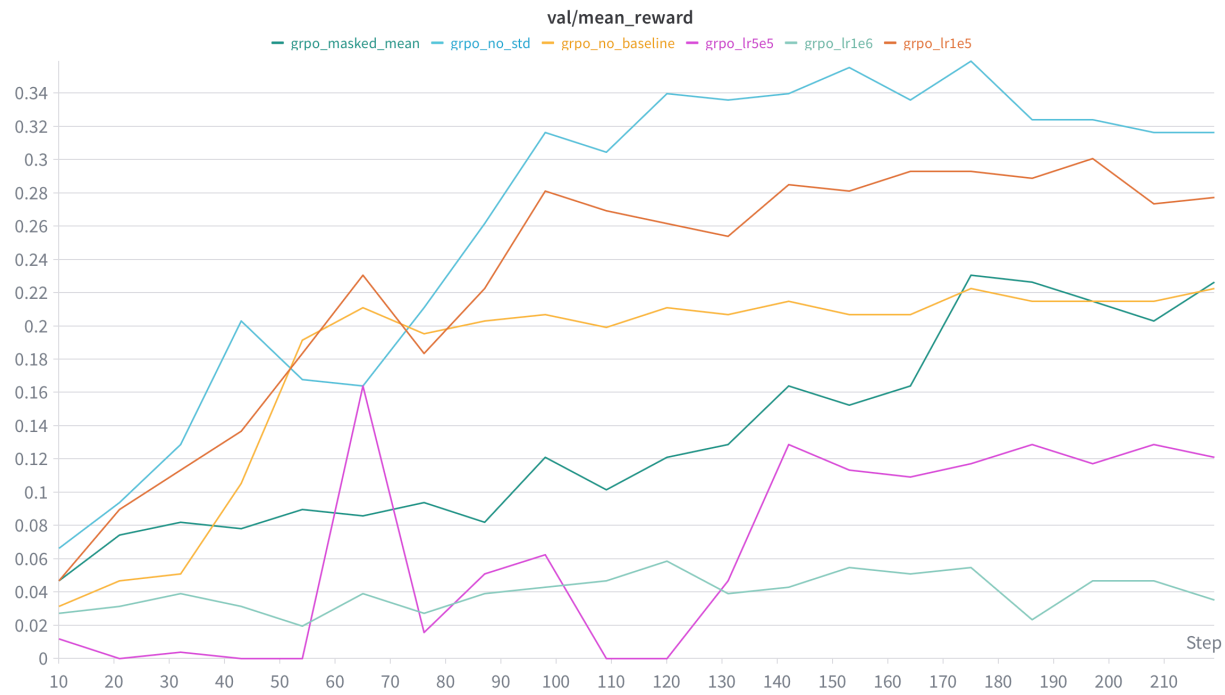
Validation reward improves steadily from ~5% at step 10 to ~30% at step 200, confirming the policy is learning. Early rollouts (step 10) show the model attempting arithmetic but ignoring the constraint that each number must be used exactly once and failing to close the </answer> tag correctly. By step 50 the model has learned the output format and begins finding correct solutions through systematic reasoning. By step 100-200, responses are consistently well-structured with clear step-by-step reasoning, though mathematical accuracy remains imperfect the model makes arithmetic errors or invents numbers not in the input list.

GRPO_LEARNING_RATE



LR=1e-5 hits 30.1% at step 180
LR=5e-5 shows rapid format reward saturation (reaching 1.0 by step 100) while answer reward collapses to near zero, indicating the policy is reward hacking by producing correct format tokens without solving the underlying task. LR=1e-6 produces near-zero gradient norms throughout training, confirming the step size is too small to meaningfully update the policy within 200 steps.

GRPO_BASELINE

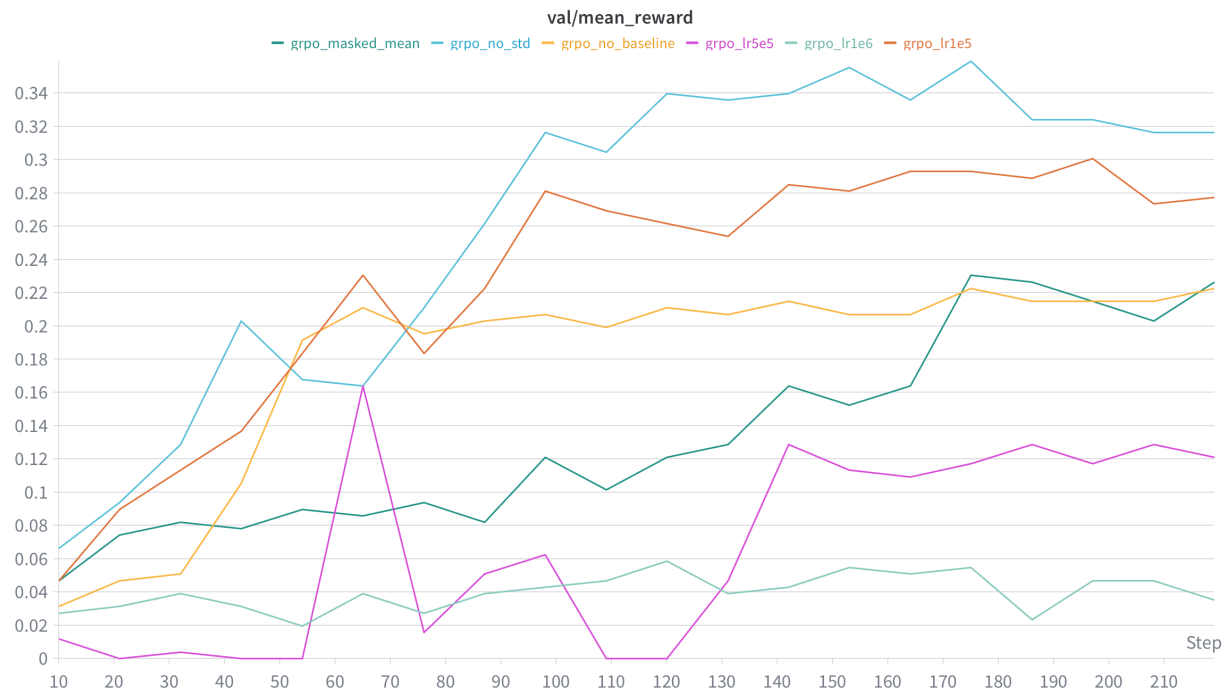


We compared reinforce_with_baseline (group-normalized advantages) against no_baseline (raw rewards as advantages), both with LR=1e-5 and use_std_normalization=True. Reinforce_with_baseline reaches ~30% peak validation reward while no_baseline plateaus at ~22%. The baseline reduces gradient variance by centering rewards within each group, which allows the policy to better distinguish relatively good from relatively bad rollouts rather than treating all positive-reward responses equally. no_baseline shows notably more stable format reward convergence, reaching ~97% format accuracy by step 50 and maintaining it throughout.

THINK_ABOUT_LENGTH_NORMALISATION

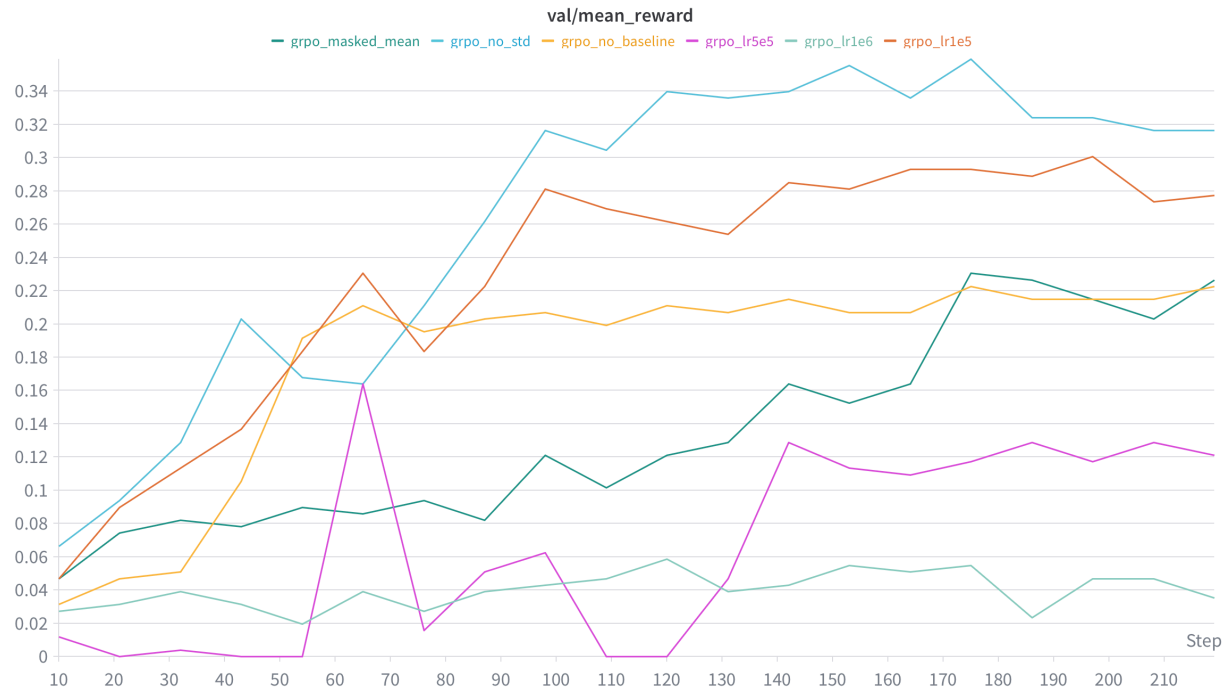
masked_mean averages the per-token loss over the unmasked response tokens. So each token contributes equally to the gradient regardless of response length, which normalizes the gradient magnitude per sequence. The downside is that it implicitly rewards shorter correct responses more than longer ones per optimizer step, which can be considered a bias in the policy
masked_normalize sums the per-token loss and divides by a fixed constant. This means shorter responses produce smaller gradient magnitudes than longer ones for the same per-token loss. If responses vary widely in length, this can lead to instability. Example a batch with one very long response and many short ones will have uneven gradient contributions. So masked_mean is preferable when response lengths vary significantly across a batch, as it stabilizes gradient magnitudes. masked_normalize is preferable when the goal is to match the theoretical policy gradient more closely or when response lengths are uniform.

GROP_LENGTH_NORMALISATION



We compared masked_normalize (default) against masked_mean, both with LR=1e-5, reinforce_with_baseline, and use_std_normalization=True.
masked_normalize outperforms masked_mean, reaching ~30% peak validation reward compared to ~23% for masked_mean. masked_mean learns more slowly in early training (10-12% at step 100 vs 27% for masked_normalize) but continues to improve gradually, suggesting it provides a weaker but more consistent signal. masked_mean shows higher gradient norm variance throughout training, consistent with the theoretical expectation.

GRPO_GROUP_STANDARD_DEVIATION



We compared use_std_normalization=True against use_std_normalization=False, both with LR=1e-5 and reinforce_with_baseline.
Removing std normalization (Dr. GRPO) outperforms the standard variant, peaking at ~36% vs ~30%. use_std_normalization=False shows more stable increasing gradient norms throughout training, whereas with_std exhibits more erratic gradient behavior especially in early training.